

Lógica Condicional Aplicada em Testes com Playwright

Funções/Propriedades úteis de **Locator** no Playwright

Aqui estão algumas das funções/metódos mais úteis de **Locator** que retornam booleanos ou valores úteis, junto com explicações baseadas na documentação oficial:

Aqui vai cada função + explicação + detalhes importantes:

Método / Propriedade	Retorno / Tipo	O que verifica / faz	Observações / cuidado
<code>locator.inner_text(**kwargs)</code>	<code>str</code>	Retorna o texto visível interno do elemento (equivalente a <code>element.innerText</code>)	Se o elemento for invisível ou removido, pode falhar. Use com cautela.
<code>locator.input_value(**kwargs)</code>	<code>str</code>	Retorna o valor atual de um <code><input></code> , <code><textarea></code> ou <code><select></code>	Só aplicável para elementos de formulário que aceitam valor.
<code>locator.is_checked(**kwargs)</code>	<code>bool</code>	Retorna <code>True</code> se o elemento for um checkbox ou rádio e estiver "checado"	Se não for checkbox/rádio, vai gerar exceção.
<code>locator.is_disabled(**kwargs)</code>	<code>bool</code>	Retorna <code>True</code> se o elemento estiver desabilitado (ou seja, não acionável)	É o oposto de "habilitado".
<code>locator.is_editable(**kwargs)</code>	<code>bool</code>	Verifica se o elemento permite edição (por exemplo, um <code><input></code> que pode ser modificado)	Útil antes de usar <code>.fill()</code> ou <code>.type()</code> .

Método / Propriedade	Retorno / Tipo	O que verifica / faz	Observações / cuidado
<code>locator.is_enabled(**kwargs)</code>	<code>bool</code>	Retorna <code>True</code> se o elemento está habilitado (não desabilitado)	Muitas ações só funcionarão quando estiver habilitado.
<code>locator.is_hidden(**kwargs)</code>	<code>bool</code>	Retorna <code>True</code> se o elemento está oculto (invisível ou <code>display: none</code>)	A documentação diz que o parâmetro <code>timeout</code> é ignorado para <code>is_hidden</code>
<code>locator.is_visible(**kwargs)</code>	<code>bool</code>	Retorna <code>True</code> se o elemento estiver visível (existe no DOM, tem dimensão, não está oculto)	Muito usada para decidir se interagir ou esperar.

Referência oficial:

<https://playwright.dev/python/docs/api/class-locator#methods>

Observações sobre "actionability" e esperas automáticas

O Playwright faz **verificações automáticas de "actionability"** quando você chama métodos de ação (como `click`, `fill` etc.). Essas verificações envolvem checar se o elemento:

1. Está visível;
2. Está estável (não está em movimento ou animação);
3. Recebe eventos (não está coberto por outro elemento);
4. Está habilitado. playwright.dev

Ou seja: ainda que você tente clicar em um elemento invisível ou desabilitado, o Playwright vai aguardar (dentro do timeout) até que ele esteja "acionável" ou lançar um erro.

Exemplos práticos com `if`, `else` e `while` usando essas condições

Exemplo A: Verificar se preço de produto é menor que Rs. 500,00 antes de adicionar ao carrinho

```
from pages.produtos import Produtos

def test_adicionar_produto_caso_preco_menor_que_500(page):
    produtos = Produtos(page)
    produtos.acessar_produtos()
    preco_produto = int(produtos.card_produto.nth(1).locator('.productinfo h2').inner_text().replace('Rs. ', ''))
    print(preco_produto)
    print(type(preco_produto))
    if preco_produto <= 500:
        produtos.adicionar_produto_ao_carrinho(indice_produto=1)
        print('Produto com preço menor que 500, FOI adicionado ao carrinho')
    else:
        print('Produto com preço maior que 500, NÃO FOI adicionado ao carrinho')
```

Exemplo B: Usar **while** excluir todos produtos do carrinho

```
from pages.carrinho import Carrinho

def test_excluir_todos_produtos_carrinho(page):
    carrinho = Carrinho(page)
    carrinho.acessar_carrinho()
    while carrinho.botao_deletar_produto.first.is_visible():
        carrinho.botao_deletar_produto.first.click()
        page.wait_for_timeout(1000)
```